# PMSCS 694
# Information Security

## Crypto: Part VI

## Lecture 8

**Md. Rafsan Jani**
**Associate Professor**
**Department of Computer Science and Engineering**
**Jahangirnagar University**

# Diffie-Hellman

# Diffie-Hellman Key Exchange

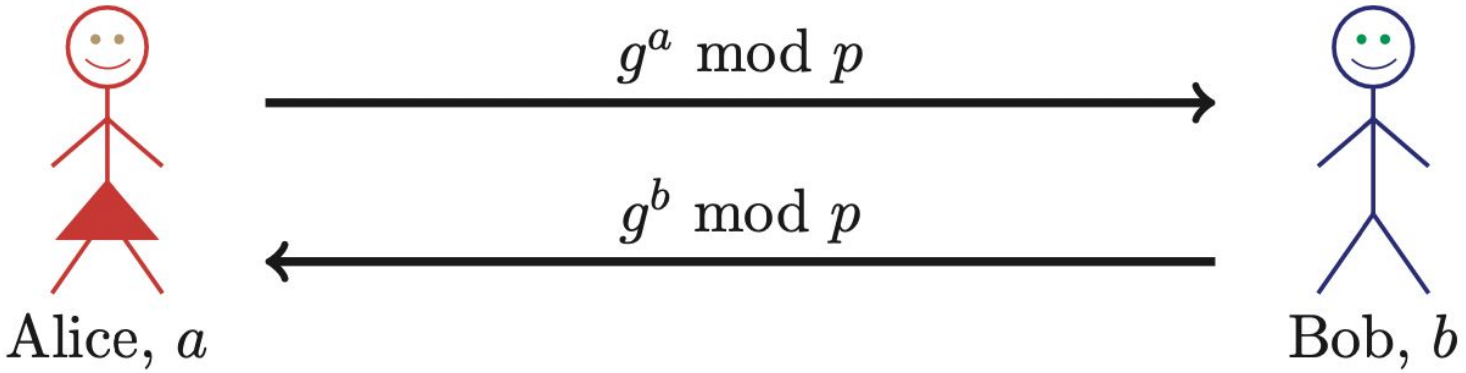
- Invented by Williamson (GCHQ) and, independently, by D and H (Stanford)
- A "key exchange" algorithm
  - Used to establish a shared symmetric key
  - ***Not*** for encrypting or signing
- Based on **discrete log** problem
  - **Given:** g, p, and $g^k \bmod p$
  - **Find:** exponent k

# Diffie-Hellman

- Let $p$ be prime, let $g$ be a **generator**
  - For any $x \in \{1,2,\ldots,p-1\}$ there is $n$ s.t. $x = g^n \bmod p$
- Alice selects her private value $a$
- Bob selects his private value $b$
- Alice sends $g^a \bmod p$ to Bob
- Bob sends $g^b \bmod p$ to Alice
- Both compute shared secret, $g^{ab} \bmod p$
- Shared secret can be used as symmetric key

# Diffie-Hellman

- **Public:** g and p
- **Private:** Alice's exponent a, Bob's exponent b



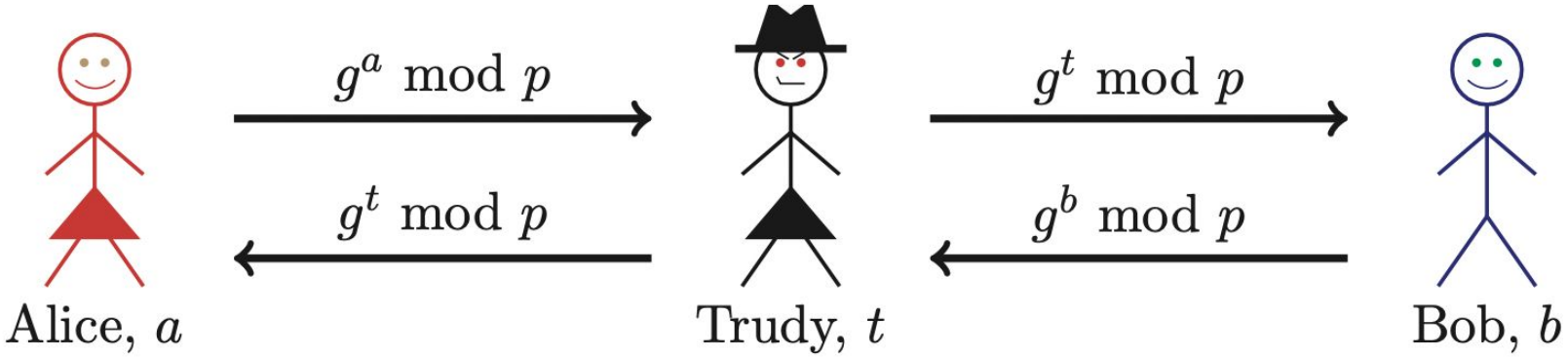

- Alice computes $(g^b)^a = g^{ba} = g^{ab} \bmod p$
- Bob computes $(g^a)^b = g^{ab} \bmod p$
- They can use $K = g^{ab} \bmod p$ as symmetric key

# Diffie-Hellman

- Suppose Bob and Alice use Diffie-Hellman to determine symmetric key $K = g^{ab} \bmod p$
- Trudy can see $g^a \bmod p$ and $g^b \bmod p$
  - But... $g^a g^b \bmod p = g^{a+b} \bmod p \neq g^{ab} \bmod p$
- If Trudy can find $a$ or $b$, she gets $K$
- If Trudy can solve **discrete log** problem, she can find $a$ or $b$

# Diffie-Hellman

- Subject to man-in-the-middle (MiM) attack




- Trudy shares secret $g^{at} \bmod p$ with Alice
- Trudy shares secret $g^{bt} \bmod p$ with Bob
- Alice and Bob don't know Trudy is MiM

# Diffie-Hellman

- How to prevent MiM attack?
  - Encrypt DH exchange with symmetric key
  - Encrypt DH exchange with public key
  - Sign DH values with private key
  - Other?
- At this point, DH may look pointless…
  - …but it's not (more on this later)
- You **MUST** be aware of MiM attack on Diffie-Hellman

# Man-in-the-Middle (MITM) Attack

- A **Man-in-the-Middle (MITM) attack** is a type of security attack in which an attacker secretly intercepts, relays, and possibly alters the communication between two parties who believe they are communicating directly with each other
- In simple terms:
  - **Alice thinks she is talking to Bob.**
  - **Bob thinks he is talking to Alice.**
  - **But Trudy (the attacker) is silently sitting in between.**

# Communication Under MITM Attack

- In a MITM attack:
  - *Alice → Trudy → Bob*
- Trudy intercepts messages from Alice.
- Trudy may **read**, **modify**, or **inject** messages.
- Trudy then forwards the manipulated message to Bob.
- Neither Alice nor Bob realizes the interception.

# Why MITM Attacks Are Dangerous

- MITM attacks can violate all three pillars of security:

| Security Property | How MITM Breaks It |
| --- | --- |
| Confidentiality | Attacker reads sensitive data |
| Integrity | Attacker alters messages |
| Authentication | Attacker impersonates users |

# Common Types of MITM Attacks

- **Passive MITM**
  - Attacker only listens.
  - No modification.
  - Hard to detect.
- **Active MITM**
  - Attacker modifies messages.
  - Can inject fake data.
  - More destructive.

# How to Prevent MITM Attacks

- **Cryptographic Countermeasures**

| Technique | Purpose |
| --- | --- |
| Digital certificates | Verify identity |
| Digital signatures | Ensure authenticity |
| TLS / HTTPS | Secure authenticated channels |
| Public Key Infrastructure (PKI) | Trust management |

# How to Prevent MITM Attacks

- **Practical Countermeasures**
- Always verify HTTPS certificates.
- Use VPNs on public Wi-Fi.
- Avoid open, unsecured networks.
- Enable mutual authentication.

# Elliptic Curve Cryptography

# Elliptic Curve Crypto (ECC)

- "Elliptic curve" is **not** a cryptosystem
- Elliptic curves provide different way to do the math in public key system
- Elliptic curve versions of DH, RSA, ...
- Elliptic curves are more efficient
  - Fewer bits needed for same security
  - But the operations are more complex, yet it is a big "win" overall

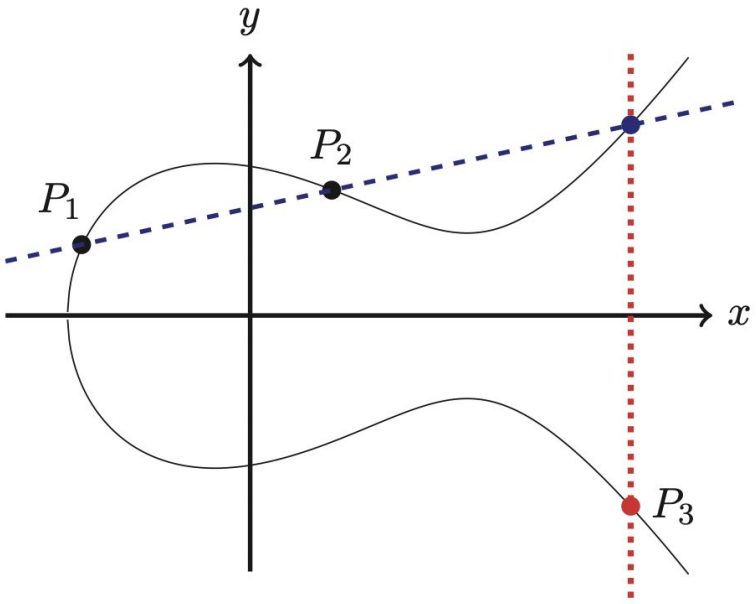
# What is an Elliptic Curve?

- An elliptic curve $E$ is the graph of an equation of the form

$$y^2 = x^3 + ax + b$$

- Also includes a "point at infinity"
- What do elliptic curves look like?
- See the next slide!

# Elliptic Curve Picture




- Consider elliptic curve

  $E: y^2 = x^3 - x + 1$

- If $P_1$ and $P_2$ are on $E$, we can define addition,

  $P_3 = P_1 + P_2$

  as shown in picture
- Addition is all we need…

# Points on Elliptic Curve

- Consider $y^2 = x^3 + 2x + 3 \pmod 5$

  $x = 0 \Rightarrow y^2 = 3 \Rightarrow$ no solution $\pmod 5$

  $x = 1 \Rightarrow y^2 = 6 = 1 \Rightarrow y = 1,4 \pmod 5$

  $x = 2 \Rightarrow y^2 = 15 = 0 \Rightarrow y = 0 \pmod 5$

  $x = 3 \Rightarrow y^2 = 36 = 1 \Rightarrow y = 1,4 \pmod 5$

  $x = 4 \Rightarrow y^2 = 75 = 0 \Rightarrow y = 0 \pmod 5$

- Then points on the elliptic curve are

  $(1,1)$ $(1,4)$ $(2,0)$ $(3,1)$ $(3,4)$ $(4,0)$

  and the point at infinity: $\infty$

# Elliptic Curve Math

- **Addition on:** $y^2 = x^3 + ax + b \pmod p$

  $P_1=(x_1,y_1)$, $P_2=(x_2,y_2)$

  $P_1 + P_2 = P_3 = (x_3,y_3)$ **where**

  $x_3 = m^2 - x_1 - x_2 \pmod p$

  $y_3 = m(x_1 - x_3) - y_1 \pmod p$

  **And** $m = (y_2-y_1)*(x_2-x_1)^{-1} \bmod p$, **if** $P_1 \neq P_2$

  $m = (3x_1^2+a)*(2y_1)^{-1} \bmod p$, **if** $P_1 = P_2$

  **Special cases:** **If** $m$ **is infinite,** $P_3 = \infty$, **and**

  $\infty + P = P$ **for all** $P$

# Elliptic Curve Addition

- Consider $y^2 = x^3 + 2x + 3 \pmod 5$.
  Points on the curve are $(1,1)$ $(1,4)$ $(2,0)$ $(3,1)$ $(3,4)$ $(4,0)$ and $\infty$
- What is $(1,4) + (3,1) = P_3 = (x_3, y_3)$?

  $m = (1-4)*(3-1)^{-1} = -3*2^{-1}$

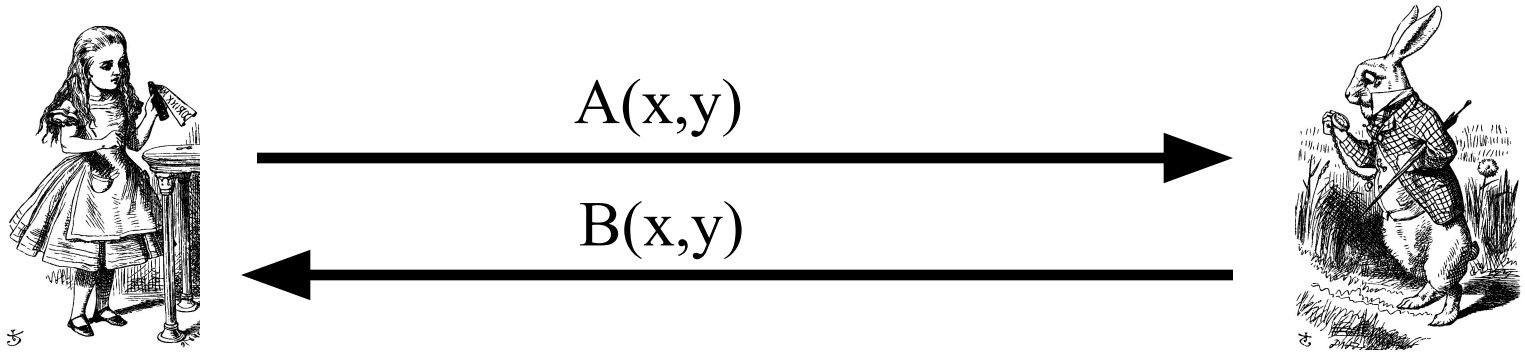
  $= 2(3) = 6 = 1 \pmod 5$

  $x_3 = 1 - 1 - 3 = 2 \pmod 5$

  $y_3 = 1(1-2) - 4 = 0 \pmod 5$
- On this curve, $(1,4) + (3,1) = (2,0)$

# ECC Diffie-Hellman

- **Public:** Elliptic curve and point $(x,y)$ on curve
- **Private:** Alice's $A$ and Bob's $B$




- Alice computes $A(B(x,y))$
- Bob computes $B(A(x,y))$
- These are the same since $AB = BA$

# ECC Diffie-Hellman

- **Public:** Curve $y^2 = x^3 + 7x + b \pmod{37}$ and point $(2,5) \Rightarrow b = 3$
- **Private:** Alice: $A = 4$, Bob: $B = 7$
- Alice sends Bob: $4(2,5) = (7,32)$
- Bob sends Alice: $7(2,5) = (18,35)$
- Alice computes: $4(18,35) = (22,1)$
- Bob computes: $7(7,32) = (22,1)$

# Larger ECC Example

- Example from Certicom ECCp-109
  - Challenge problem, solved in 2002
- Curve E: $y^2 = x^3 + ax + b \pmod{p}$
- Where

  p = 564538252084441556247016902735257

  a = 321094768129147601892514872825668

  b = 430782315140218274262276694323197
- Now what?

# ECC Example

- The following point P is on the curve E

**(x,y) = (973390109870590665231561339089 35, 1496703728461692857606823719788 98)**

- Let k = 28118384031160194966820795453 0684
- The kP is given by

**(x,y) =** (44646769697405861057630861884284, 52296809889578588047540374779097)

- And this point is also on the curve E

# Really Big Numbers!

- Numbers are big, but not big enough
  - ECCp-109 bit (32 digit) solved in 2002
- Today, ECC DH needs bigger numbers
- But RSA needs ***way*** bigger numbers
  - Minimum RSA modulus today is 1024 bits
  - That is, more than 300 decimal digits
  - That's about 10x the size of ECC example
  - And 2048 bit RSA modulus is common…

# Uses for Public Key Crypto

# Notation Reminder

- Public key notation
  - Sign $M$ with Alice's **private key**

    $[M]_{Alice}$
  - Encrypt $M$ with Alice's **public key**

    $\{M\}_{Alice}$
- Symmetric key notation
  - Encrypt $P$ with **symmetric key** $K$

    $C = E(P,K)$
  - Decrypt $C$ with **symmetric key** $K$

    $P = D(C,K)$

# Uses for Public Key Crypto

- Confidentiality
  - Transmitting data over insecure channel
  - Secure storage on insecure media
- Authentication protocols (later)
- Digital signature
  - Provides integrity and **non-repudiation**
  - No non-repudiation with symmetric keys

# Non-non-repudiation

- Alice orders 100 shares of stock from Bob
- Alice computes **MAC** using symmetric key
- Stock drops, Alice claims she did ***not*** order
- Can Bob prove that Alice placed the order?
- **No!** Bob also knows the symmetric key, so he could have forged the **MAC**
- **Problem:** Bob knows Alice placed the order, but he can't prove it

# Non-repudiation

- Alice orders 100 shares of stock from Bob
- Alice **signs** order with her private key
- Stock drops, Alice claims she did not order
- Can Bob prove that Alice placed the order?
- **Yes!** Alice's private key used to sign the order —only Alice knows her private key
- This assumes Alice's private key has not been lost/stolen

# Public Key Notation

- **Sign** message $M$ with Alice's **private key:** $[M]_{Alice}$
- **Encrypt** message $M$ with Alice's **public key:** $\{M\}_{Alice}$
- Then

$$\{[M]_{Alice}\}_{Alice} = M$$
$$[\{M\}_{Alice}]_{Alice} = M$$

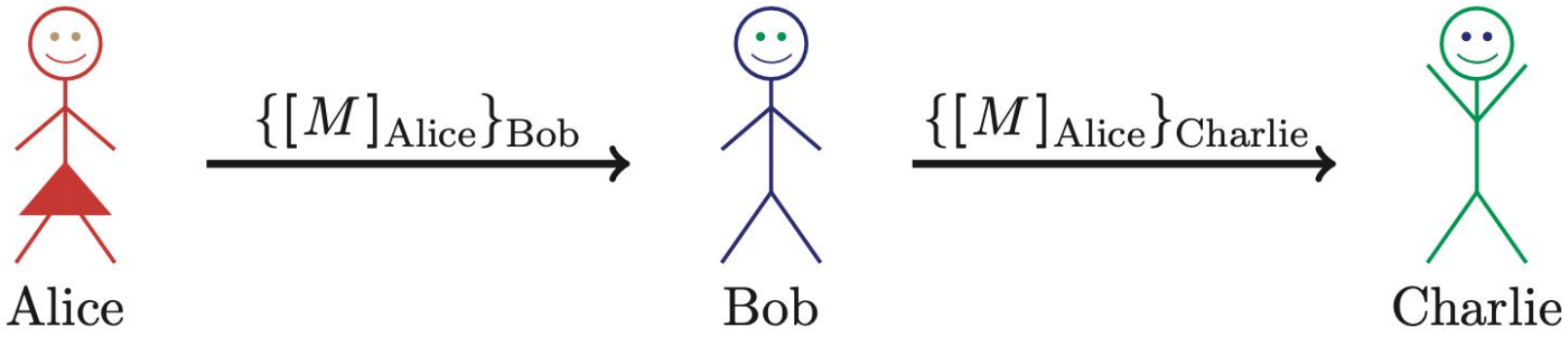
# Sign and Encrypt
# vs
# Encrypt and Sign

# Confidentiality and Non-repudiation?

- Suppose that we want confidentiality and integrity/non-repudiation
- Can public key crypto achieve both?
- Alice sends message to Bob
  - **Sign and encrypt:** $\{[M]_{Alice}\}_{Bob}$
  - **Encrypt and sign:** $[\{M\}_{Bob}]_{Alice}$
- Can the order possibly matter?

# Sign and Encrypt

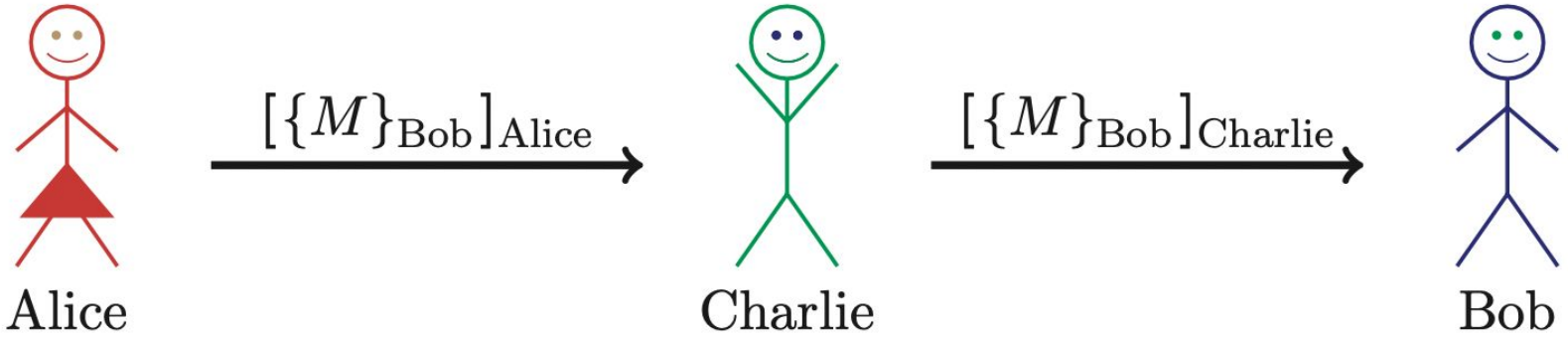
- M = "I love you"




- **Q:** What's the problem?
- **A:** No problem —public key is public

# Encrypt and Sign

- M = "My theory, which is mine…."




- **Note** that Charlie cannot decrypt M
- **Q:** What is the problem?
- **A:** No problem —public key is public

# Public Key Infrastructure

# Public Key Certificate

- Digital **certificate** contains name of user and user's public key (possibly other info too)
- It is ***signed*** by the issuer, a ***Certificate Authority*** (*CA*), such as VeriSign

  $M = (\text{Alice, Alice's public key}),\ S = [M]_{CA}$

  **Alice's Certificate** $= (M, S)$
- Signature on certificate is verified using *CA*'s public key

  Must verify that $M = \{S\}_{CA}$

# Certificate Authority

- Certificate authority (CA) is a trusted 3rd party (TTP) —creates and signs certificates
- Verify signature to verify **integrity** & identity of **owner of corresponding private key**
  - Does **not** tell us anything about the identity of the sender of certificate —certificates are public!
- Big problem if CA makes a mistake
  - CA once issued Microsoft cert. to someone else
- A common format for certificates is X.509

# PKI

- Public Key Infrastructure (PKI): The stuff needed to securely use public key crypto
  - Key generation and management
  - Certificate authority (CA) or authorities
  - Certificate revocation lists (CRLs), etc.
- No general standard for PKI
- We mention 3 generic "trust models"
  - We only discuss the CA (or CAs)

# PKI Trust Models

- Monopoly model
  - One universally trusted organization is the *CA* for the known universe
  - Big problems if *CA* is ever compromised
  - Who will act as *CA* ???
    - System is useless if you don't trust the *CA*!

# PKI Trust Models

- Oligarchy
  - Multiple (as in, "a few") trusted CAs
  - This approach is used in browsers today
  - Browser may have 80 or more CA certificates, just to verify certificates!
  - User can decide which CA or CAs to trust

# PKI Trust Models

- Anarchy model
  - Everyone is a CA…
  - Users must decide who to trust
  - This approach used in PGP: "Web of trust"
- Why is it anarchy?
  - Suppose certificate is signed by Frank and you don't know Frank, but you do trust Bob and Bob says Alice is trustworthy and Alice vouches for Frank. Should you accept the certificate?
- **Many** other trust models/PKI issues

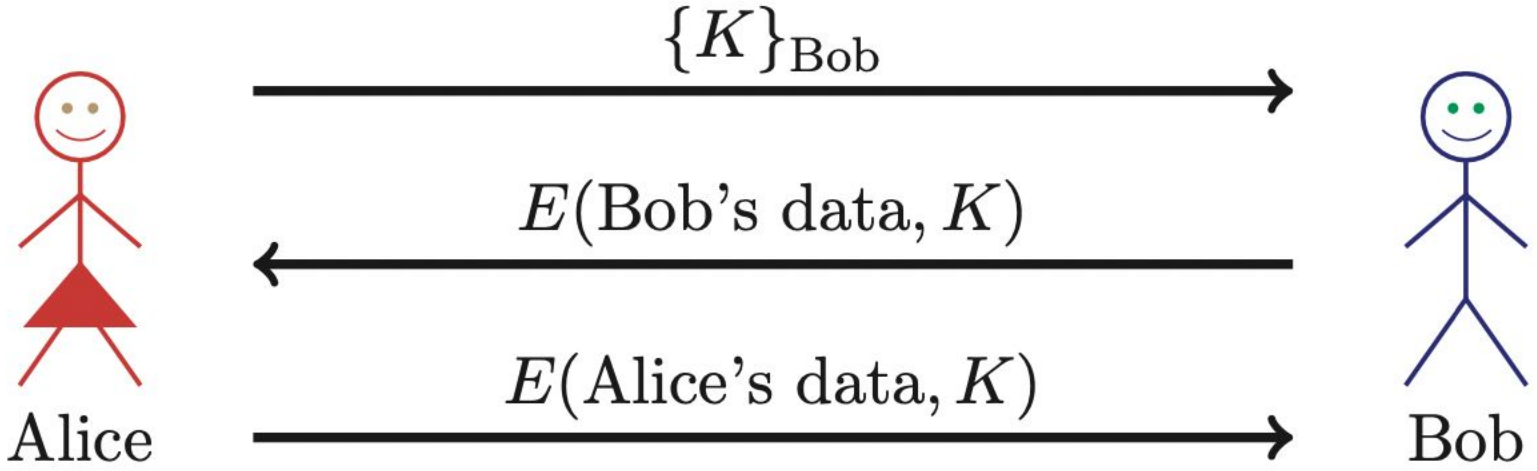
# Confidentiality in the Real World

# Symmetric Key vs Public Key

- Symmetric key +'s
  - **Speed**
  - No public key infrastructure (PKI) needed (but have to generate/distribute keys)
- Public Key +'s
  - **Signatures** (non-repudiation)
  - No ***shared*** secret (but, do have to get private keys to the right user…)

# Real World Confidentiality

- **Hybrid cryptosystem**
  - Public key crypto to establish a key
  - Symmetric key crypto to encrypt data…




- Can Bob be sure he's talking to Alice?

# Sample Questions

- *A digital signature provides for data integrity and a MAC provides for data integrity. Why does a digital signature also provides for non-repudiation while a MAC does not?*
- *Consider the elliptic curve E : y2 =x3 + 7x + b (mod 11).*
  - *a. Determine b so that the point P = (4,5) is on the curve E.*
  - *b. Using the b found in part a, list all points on E.*
  - *c. Using the b found in part a, find the sum (4,5) + (5,4) on E.*
  - *d. Using the 6 found in part a, find the point 3(4,5).*